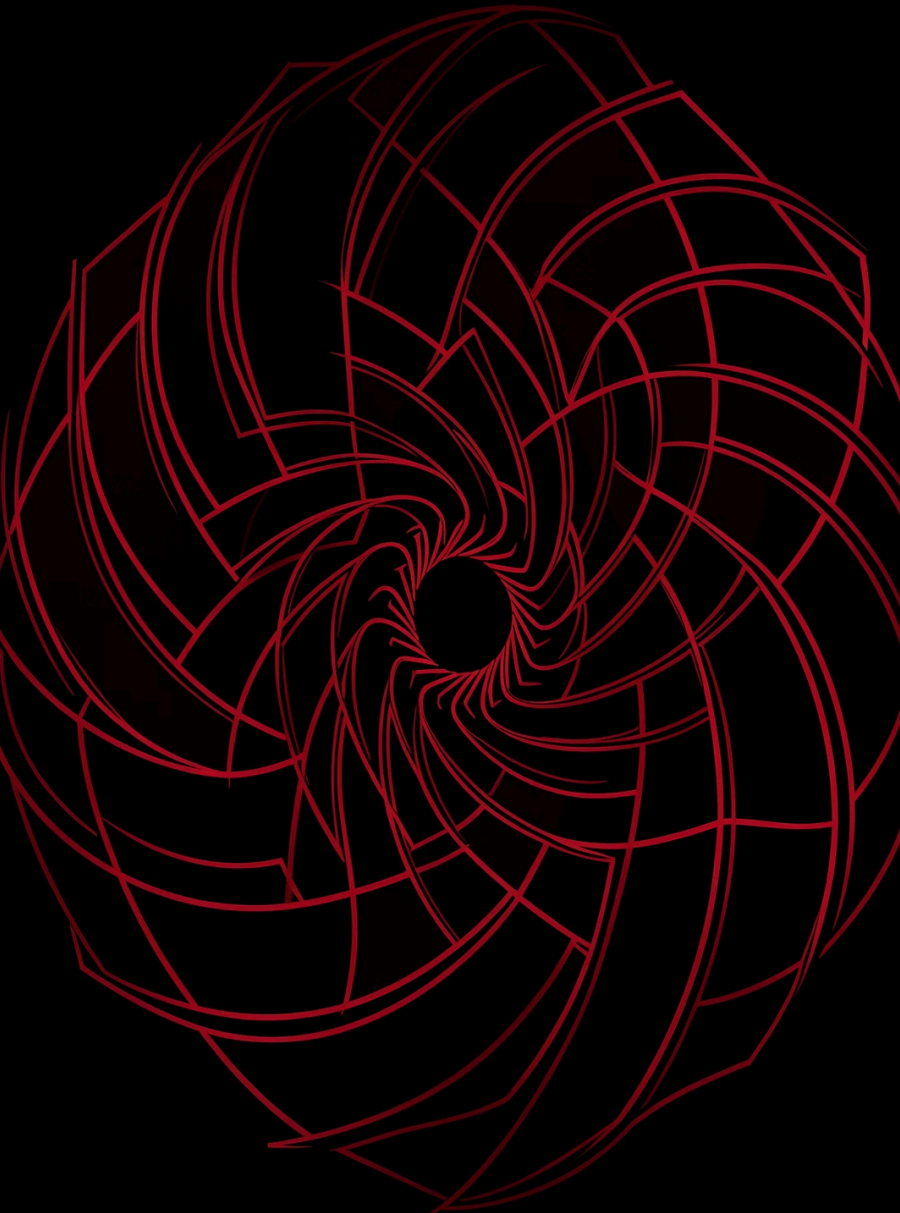




Function and Recursion

Building Blocks of Code — how a function that calls itself unlocks elegant solutions to complex problems.

What is Recursion?

Recursion is a concept where a definition or process **refers to itself**. In computing, a **recursive function** is one that calls itself to solve a smaller version of the same problem.

Nesting Dolls

Each doll contains a smaller version of itself inside

Fractals

Nature's self-repeating patterns at every scale

File Systems

Folders nested inside folders, infinitely



The Two Pillars of Recursion

Every recursive function rests on two essential components. Without both, recursion either never starts — or never stops.

Base Case

The simplest version of the problem. The function does **not** call itself here. This is the **stopping condition** that prevents infinite loops.

Recursive Case

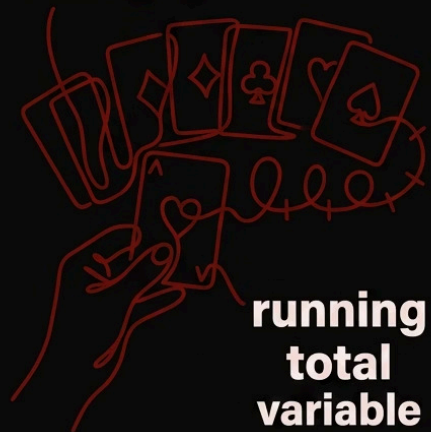
The problem is broken into a **smaller, similar sub-problem**. The function calls itself with this reduced input, steadily marching toward the base case.

Iteration vs. Recursion

Adding up a deck of cards — two different mental models for the same result.

ITERATIVE APPROACH

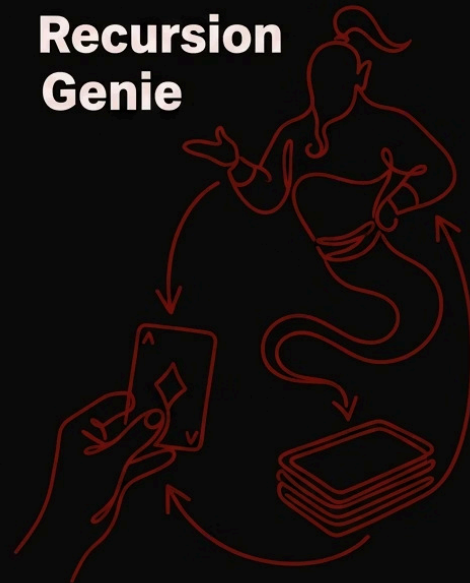
one by one loop
variable



one by one
loop

RECURSIVE APPROACH

Recursion
Genie



delegates rest,
then adds your card

The iterative approach is direct and familiar. The recursive approach is more abstract — but it mirrors how we naturally decompose problems: **solve one piece, trust the rest.**

Visualizing Recursion: `countDown`

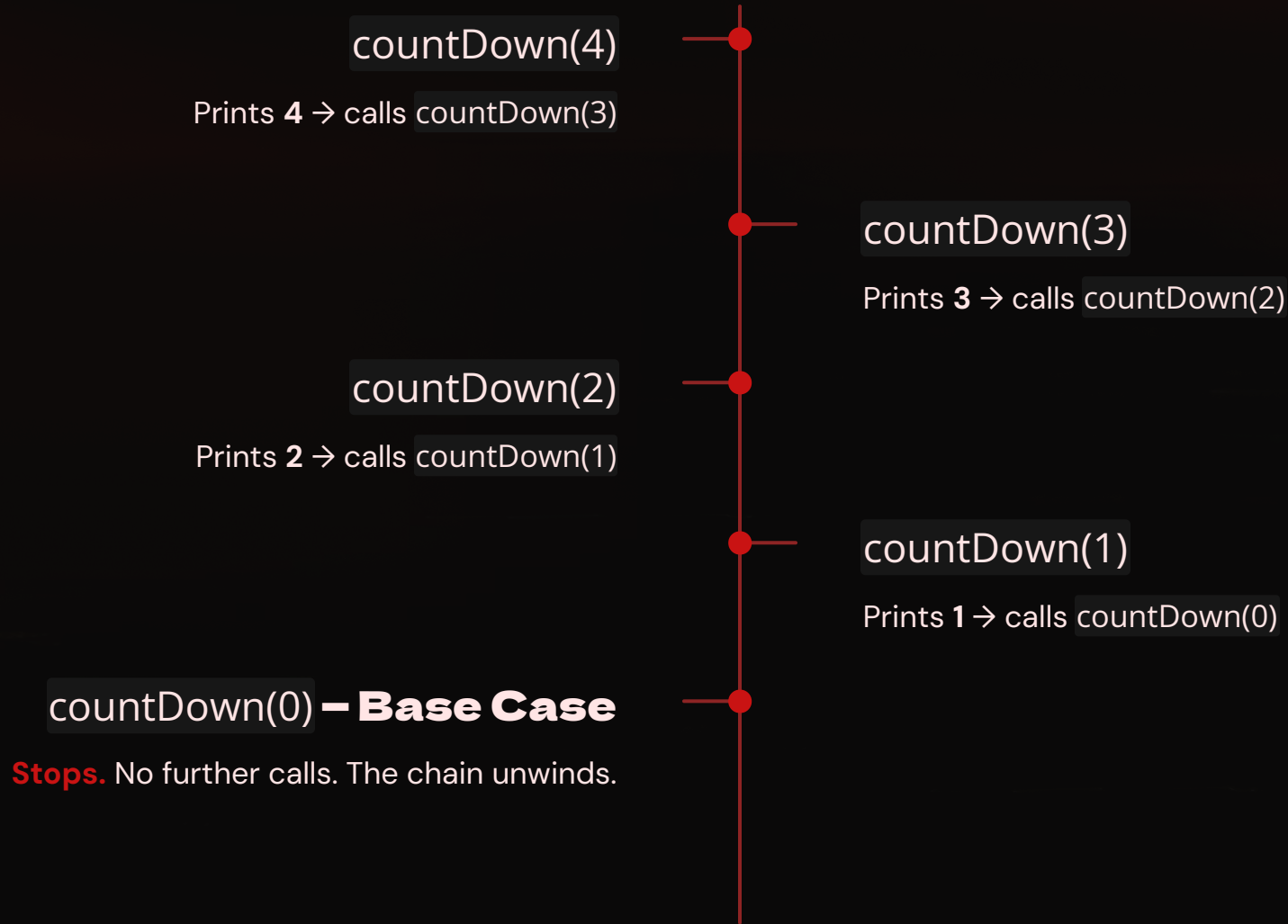
Print numbers from `n` down to 1 — **without any loops**.



The recursive step reduces `n` by 1 each call, steadily approaching the base case that halts execution.

countDown in Action: Step-by-Step Trace

Calling `countDown(4)` creates a chain of self-calls — each one waiting for the next to return.



The Power of Simple Rules

Recursion can generate **complex, beautiful patterns** from remarkably simple instructions. A spiral function draws a line, turns, and calls itself with a slightly smaller length — repeating until the lines become too small to draw.



The spiral Function

`spiral(sideLen, angle, scaleFactor, minLength)`

Recursive Case

While `sideLen ≥ minLength`:

- Draw a line of `sideLen`
- Turn by `angle`
- Call `spiral(sideLen × scaleFactor)`

Base Case

When `sideLen < minLength`, the recursion stops — the drawing is complete.

Why Use Recursion?



Elegance

Solves complex problems with remarkably simple, clean algorithms that mirror the problem's own structure.



Data Structures

Opens the door to trees, graphs, and nested structures — naturally expressed through recursion.



Natural Decomposition

For problems that break into smaller identical sub-problems, recursion is the most intuitive approach.

Recursion: A Fundamental Concept

Mastering recursion is more than a coding exercise — it's a shift in how you **think about problems**.

Unlock Advanced Algorithms

Sorting, searching, and dynamic programming all build on recursive thinking

New Problem-Solving Lens

See problems as self-similar structures waiting to be decomposed

Foundation for What's Next

Trees, graphs, divide-and-conquer — recursion is the gateway

